

#8 Depth First Search, Backtracking

김성수

4thrunyul@gmail.com

0. 목차



1. Depth First Search

- Graph
- Grid Graph
- State Space Tree

2. Backtracking

1. Depth First Search

1. Depth First Search

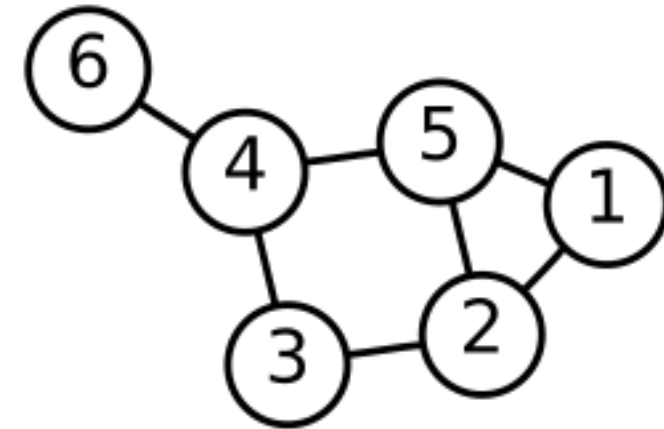
Depth First Search(DFS, 깊이 우선 탐색)

- 가능한 모든 경우를 빠짐 없이 탐색하는 Brute Force(완전 탐색) 알고리즘의 한 종류
- 하나의 탐색이 끝날 때까지 멈추지 않고 탐색하는 방식
- 막다른 상태(기저 조건)에 도달하면 '아직 탐색하지 않은 선택지가 남은 직전 상태'로 돌아가서 다른 경로 탐색
- 후입선출 구조이며 주로 재귀나 스택을 활용해 구현
- 그래프, 격자 그래프, 상태 공간 등 문제 유형이 많음

1. Depth First Search: Graph

Graph Basics

- Vertex / Node(정점 / 노드): 위치나 데이터를 나타내는 단위
 - $V = \{1, 2, 3, 4, 5, 6\}$
- Edge(간선): 정점과 정점 사이를 잇는 연결선
 - $E = \{(1, 2), (1, 5), (2, 3), (2, 5), (3, 4), (4, 5), (4, 6)\}$
- Adjacent / Neighbor(인접 정점): 간선으로 직접 연결된 정점
 - 정점 4의 인접 정점은 3, 5, 6
- Degree(차수): 한 정점에 연결된 간선의 수
 - 정점 5의 차수는 3, 정점 6의 차수는 1
- Path(경로): 간선을 따라 정점을 순서대로 이동하는 경로
 - ex: $6 \rightarrow 4 \rightarrow 5 \rightarrow 1$



1. Depth First Search: Graph



Depth First Search: Graph

- 그래프의 한 정점에서 출발하여 갈 수 있는 가장 깊은 곳까지 탐색
- 현재 정점과 연결된 인접 정점 중 아직 방문하지 않은 곳으로 재귀 호출 진행
- 더 이상 갈 수 있는 정점이 없으면 직전 정점으로 복귀

1. Depth First Search: Graph

Depth First Search: Graph

- `vector<int> adj[101]` // 인접 리스트
 - `vector<int>` 101개로 구성된 1차원 배열
 - 각 정점 번호(인덱스)마다 연결된 인접 정점 목록을 동적으로 저장
- `bool visit[101]` // 방문 노드 체크 배열
 - 각 정점의 방문 여부 기록
 - 이미 방문한 정점의 재방문을 차단하여 무한 루프 방지
- `adj[u].push_back(v), adj[v].push_back(u)`
 - 양방향 그래프 표현을 위해 $u \rightarrow v, v \rightarrow u$ 양방향 경로 모두 등록
- `sort(adj[i].begin(), adj[i].end())`
 - 한 정점에서 이동 가능한 다음 정점이 여러 개일 때 오름차순으로 정렬
 - 번호가 작은 정점부터 먼저 방문하도록 탐색 순서 보장
 - 꼭 필요한 로직은 아니고 문제마다 다름

1. Depth First Search: Graph

Depth First Search: Graph

- 1. 방문 처리
 - 현재 정점에 도달하자마자 방문 완료 표시
 - `visit[cur] = true`
- 2. 인접 정점 순회
 - 현재 정점과 연결된 정점 목록(`adj[cur]`)을 순차적으로 확인
 - `for(int next : adj[cur])`
- 3. 깊이 우선 탐색
 - 아직 방문하지 않은 인접 정점 발견 시 재귀 호출을 통해 다음 노드로 이동
 - `if(!visit[next]) dfs(next);`
- 4. 복귀
 - 더 방문할 곳(미방문 인접 정점)이 없으면 함수가 종료되며 직전 호출 위치로 자동 복귀
 - 이후 다음 분기 탐색 재개

1. Depth First Search: Graph

문제

- [1818 바이러스](#)
- [코드](#)

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  // vector<int>를 원소로 가지는 배열 101개
8  vector<int> adj[101];
9  bool visit[101];
10
11 void dfs(int cur) {
12     visit[cur] = true;
13
14     for(int next : adj[cur]) {
15         if(!visit[next]) dfs(next);
16     }
17 }
```

```
19 int main() {
20     ios_base::sync_with_stdio(false);
21     cin.tie(NULL);
22
23     int n, m, u, v;
24     // u, v : 노드
25     int cnt = 0;
26
27     cin >> n >> m;
28
29     for(int i = 1; i <= m; i++) {
30         cin >> u >> v;
31
32         adj[u].push_back(v);
33         adj[v].push_back(u);
34     }
```

```
35
36     for(int i = 1; i <= n; i++) {
37         sort(adj[i].begin(), adj[i].end());
38     }
39
40     dfs(1);
41
42     for(int i = 2; i <= n; i++) {
43         if(visit[i]) ++cnt;
44     }
45
46     cout << cnt;
47
48     return 0;
49 }
```

1. Depth First Search: Graph

문제

- [1818 바이러스](#)
- [코드](#)

정렬된 인접리스트

adj[1] : [1, 5]
adj[2] : [1, 3, 5]
adj[3] : [2]
adj[4] : [7]
adj[5] : [1, 2, 6]
adj[6] : [5]
adj[7] : [4]

1. dfs(1) 실행

- 1번 노드 방문 처리 (visit[1] = true)
- adj[1]를 참고하여 dfs(2)와 dfs(5) 실행 준비

1-1. dfs(2) 실행

- 2번 노드 방문 처리
- adj[2]를 참고하여 dfs(3)과 dfs(5) 실행 준비
- 이때 1번 노드는 visit[1] = true 처리 되었으므로 실행하지 않음

1-1-1. dfs(3) 실행

- 3번 노드 방문 처리
- 2번 노드가 방문 처리 되었고 다른 연결된 노드가 없으므로 1-1로 복귀

1-1-2. 1-1 복귀 후 dfs(5) 실행

- 5번 노드 방문 처리
- adj[5]를 참고하여 dfs(6) 실행 준비

1-1-2-1. dfs(6) 실행

- 6번 노드 방문 처리
- 방문된 노드 외에 다른 노드가 없으므로 1-1-2 복귀
- 1-1-2에서도 남은 노드가 없으므로 1-1 복귀
- 1-1에서 준비 중이던 dfs(5)는 이미 방문 처리 되었고 남은 노드가 없으므로 1로 복귀
- 1에서 준비 중이던 dfs(5)는 이미 방문 처리 되었고 남은 노드가 없으므로 함수 종료

1. Depth First Search: Graph



실습

- [1818 바이러스](#)
 - [코드](#)
- [5913 친구의 친구는 친구](#)
- [1906 촌수계산](#)

1. Depth First Search: Grid Graph



Depth First Search: Grid Graph

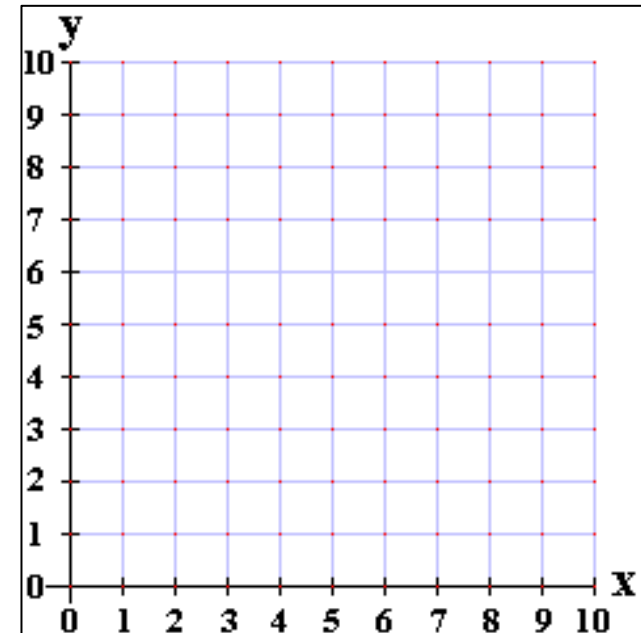
- 2차원 격자의 각 칸을 하나의 정점으로 정의 (행·열(r, c) / 좌표(x, y))
- 상하좌우 이동 처리를 위해 방향 배열(dr, dc / dx, dy) 활용
- 격자 범위(인덱스 초과), 장애물(벽), 방문 여부를 순서대로 검사하며 재귀 탐색

1. Depth First Search: Grid Graph

Grid Graph에서 주의점

- 배열 인덱스와 수학적 좌표를 헷갈리면 안 됨
- 행·열 방식에서는 row(행)은 세로, column(열)은 가로로 정의되기에 $\text{grid}[r][c]$ 으로 바로 매핑 가능
- 좌표 방식에서는 x 를 가로, y 를 세로로 정의하기에 $\text{grid}[y][x]$ 로 매핑하고 접근해야 함
- 하지만 본인이 일관적으로 x 를 행, y 를 열로 규정하고 모든 로직에서 일관성을 유지하면 $\text{grid}[x][y]$ 사용 가능

	Column →				
Row ↓	(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
	(1,0)	(1,1)	(1,2)	(1,3)	(1,4)
	(2,0)	(2,1)	(2,2)	(2,3)	(2,4)
	(3,0)	(3,1)	(3,2)	(3,3)	(3,4)
	(4,0)	(4,1)	(4,2)	(4,3)	(4,4)



1. Depth First Search: Grid Graph



Depth First Search: Grid Graph

- `grid[n][m]` // 2차원 격자 맵
 - 각 격자 칸의 상태 정보(지형, 벽, 가중치, 색상 등)를 저장
 - 자료형을 bool로 둘지 int로 둘지는 문제에 따라 다름
- `bool visit[n][m]` // 2차원 방문 체크 배열
 - 각 칸의 방문 여부 기록 및 재방문 차단
- `dr[] = {-1, 1, 0, 0}`, `dc[] = {0, 0, -1, 1}` // 행·열 방향 배열
 - 현재 위치 기준 상·하·좌·우 인접 칸으로 이동하기 위한 행과 열의 변화량 정의
 - 반복문 하나로 4방향 이동을 간결하게 처리

1. Depth First Search: Grid Graph



Depth First Search: Grid Graph

- 1. 방문 처리
 - 현재 칸(r, c)에 도달하자마자 방문 완료 표시
 - `visit[r][c] = true;`
- 2. 인접 칸 순회(방향 탐색)
 - `for(int i = 0; i < 4; i++) {`
 - `int nr = r + dr[i];`
 - `int nc = c + dc[i];`
 - `}`
 - * `dr[] = {-1, 1, 0, 0}`
 - * `dc[] = {0, 0, -1, 1}`

1. Depth First Search: Grid Graph



Depth First Search: Grid Graph

- 3. 유효성 검사 및 깊이 우선 탐색
 - 격자 범위 내, 이동 가능, 미방문 조건을 모두 만족하면 재귀 호출을 통해 다음 칸으로 이동
 - `if(0 <= nr && nr < n && 0 <= nc && nc < m) { // 격자 범위 내 인지 체크`
 - `if(grid[nr][nc] == 1 && !visit[nr][nc]) dfs(nr, nc); // 이동 가능한지, 아직 방문 안했는지 체크`
 - `}`
- 4. 복귀
 - 더 방문할 유효한 인접 칸이 없으면 함수가 종료되며 직전 호출 위치로 자동 복귀
 - 이후 다음 방향 탐색 재개

1. Depth First Search: Grid Graph



문제

- [1154 빨강과 검정](#)
- 코드

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int w, h;
7  int cnt = 0;
8  bool board[21][21];
9  bool visit[21][21];
10 int dr[] = {-1, 1, 0, 0};
11 int dc[] = {0, 0, -1, 1};
```

```
13 void dfs(int r, int c) {
14     visit[r][c] = true;
15     ++cnt;
16
17     for(int i = 0; i < 4; ++i) {
18         int nr = r + dr[i];
19         int nc = c + dc[i];
20
21         if(nr >= 1 && nr <= h && nc >= 1 && nc <= w) {
22             if(board[nr][nc] == 1 && visit[nr][nc] == false) dfs(nr, nc);
23         }
24     }
25 }
```

```
int main() {
    int r, c;
    string s;

    cin >> w >> h;

    for(int i = 1; i <= h; ++i) {
        cin >> s;

        for(int j = 1; j <= w; ++j) {
            if(s[j - 1] == '.') board[i][j] = 1;
            else if(s[j - 1] == '#') board[i][j] = 0;
            else if(s[j - 1] == '@') {
                board[i][j] = 1;
                r = i;
                c = j;
            }
        }

        dfs(r, c);

        cout << cnt;

        return 0;
    }
```

1. Depth First Search: Grid Graph



실습

- [1154 빨강과 검정](#)
 - 코드
- [1695 단지번호붙이기](#)
 - 코드

1. Depth First Search: Grid Graph

문제

- [1695 단지번호붙이기](#)
- [코드](#)

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5
6  using namespace std;
7
8  int board[26][26];
9  bool visit[26][26];
10 int n;
11 // 방향성 : 상, 하, 좌, 우
12 int dr[] = {-1, 1, 0, 0};
13 int dc[] = {0, 0, -1, 1};
14
15 int dfs(int row, int col) {
16     visit[row][col] = true;
17
18     int cnt = 1;
19
20     for(int i = 0; i < 4; ++i) {
21         int nr = row + dr[i];
22         int nc = col + dc[i];
23
24         if(nr >= 0 && nr < n && nc >= 0 && nc < n) {
25             if(board[nr][nc] == 1 && visit[nr][nc] == false) cnt += dfs(nr, nc);
26         }
27     }
28
29     return cnt;
30 }
```

```
32 int main() {
33     string s;
34     vector<int> v;
35
36     cin >> n;
37
38     for(int i = 0; i < n; ++i) {
39         cin >> s;
40
41         for(int j = 0; j < n; ++j) {
42             if(s[j] == '1') board[i][j] = 1;
43         }
44     }
45
46     for(int i = 0; i < n; ++i) {
47         for(int j = 0; j < n; ++j) {
48             if(board[i][j] == 1 && visit[i][j] == false) {
49                 v.push_back(dfs(i, j));
50             }
51         }
52     }
```

```
54     sort(v.begin(), v.end());
55
56     cout << v.size() << "\n";
57
58     for(int i : v) {
59         cout << i << "\n";
60     }
61
62     return 0;
63 }
```

1. Depth First Search: State Space Tree



Depth First Search: State Space Tree

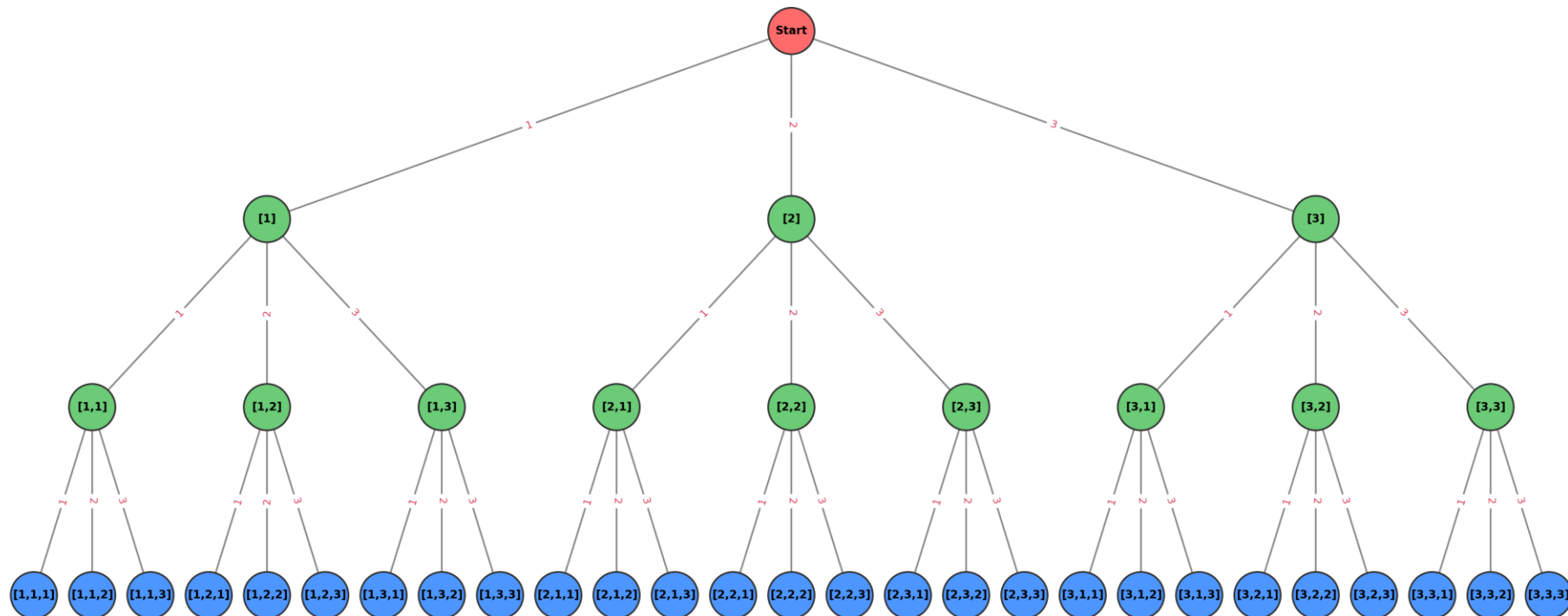
- 문제의 '현재 상태(변수 값)'를 노드로, '선택 가능한 분기'를 간선으로 정의
- 재귀 함수의 인자(depth, sum, idx 등)를 통해 현재 상태를 다음 분기로 전달
- 다음 분기 탐색 전 상태 갱신 및 탐색 완료 후 원래대로 상태 복구
- n 개 중 m 개를 고르는 조합 등 모든 경우의 수를 트리 형태로 탐색

1. Depth First Search: State Space Tree

문제

- [1169 주사위 던지기1](#)
- [코드](#)

State Space Tree: Dice Thrown 3 Times (Branching Factor = 3)



1. Depth First Search: 상태 공간



실습

- [1169 주사위 던지기1](#)
- [1513 팔씨름 대회](#)
- [1027 좋은수열](#)

2. Backtracking

2. Backtracking



Backtracking(백트래킹)

- 문제를 해결할 수 있는 모든 경로를 탐색하되, 선택한 경로가 유효하지 않거나 조건에 만족하는 해를 찾지 못할 경우 이전 단계로 되돌아가 다른 경로를 탐색하는 알고리즘
- 브루트 포스 방식에 효율성을 더한 기법
- 재귀 함수를 통해 구현할 수 있음
- Pruning(가지 치기)
 - 현재 노드에서 원하는 목표(해)에 도달할 수 있는지 확인
 - 도달할 수 없다고 판단되면 해당 경로를 더 이상 판단하지 않고 이전 노드로 돌아 감

2. Backtracking



저번주에 풀었던 문제를 잠깐 떠올려 봅시다...

- [1169 주사위 던지기1](#)
- [1175 주사위 던지기2](#)

2. Backtracking

- 1169 주사위 던지기1
- 코드

```
8  bool check[7];  
  
46 void dice3(int cnt) {  
47     if(cnt == n) {  
48         for(int i = 0; i < n; ++i) {  
49             cout << store[i] << " ";  
50         }  
51  
52         cout << "\n";  
53  
54         return;  
55     }  
56  
57     for(int i = 1; i <= 6; ++i) {  
58         if(check[i]) continue;  
59  
60         check[i] = true;  
61         store[cnt] = i;  
62  
63         dice3(cnt + 1);  
64  
65         check[i] = false;  
66     }  
67 }
```

type = 3 : 주사위를 N번 던져서 모두 다른 수가 나올 수 있는 모든 경우

Pruning

- if(check[i]) continue; 에서 가지 치기가 발생함
- check[i] = true; 에서 현재 해당 수를 점유를 설정
- check[i] = false; 에서 재귀 호출이 끝나고 다음 선택을 위해 복구
- 가지치기를 하지 않고 수열이 완성된 상태(ex. {1, 2, 3}, {2, 4, 2} 등)에서 같은 수가 아닌지 확인하는 방법도 존재하지만 효율적인 방법은 아님

2. Backtracking

- 1169 주사위 던지기1
 - 코드

```
28 void dice2(int cnt, int start) {
29     if(cnt == n) {
30         for(int i = 0; i < n; ++i) {
31             cout << store[i] << " ";
32         }
33
34         cout << "\n";
35
36         return;
37     }
38
39     for(int i = start; i <= 6; ++i) {
40         store[cnt] = i;
41
42         dice2(cnt + 1, i);
43     }
44 }
```

type = 2 : 주사위를 N번 던져서 중복이 되는 경우를 제외하고 나올 수 있는 모든 경우

* 중복의 예

1 1 2 와 중복 : 1 2 1, 2 1 1

1 2 3 과 중복 : 1 3 2, 2 1 3, 2 3 1, 3 1 2, 3 2 1

Pruning

- for(int i = start; i <= 6; ++i) 에서 가지 치기가 발생함
- int i start; 가 탐색의 최적화를 도와주고 있음

2. Backtracking

- 1175 주사위 던지기2
- 코드

```
9 void dice(int cnt, int m) {
10     if((n - cnt) * 6 < m) return;
11     else if((n - cnt) * 1 > m) return;
12     else if(cnt == n) {
13         for(int i = 0; i < n; ++i) {
14             cout << store[i] << " ";
15         }
16         cout << "\n";
17         return;
18     }
19     for(int i = 1; i <= 6; ++i) {
20         store[cnt] = i;
21         dice(cnt + 1, m - i);
22     }
23 }
```

문제

한국어 Tt

자연수 N과 M을 입력 받아서 주사위를 N번 던져서 나온 눈의 합이 M이 나올 수 있는 모든 경우를 출력하는 프로그램을 작성하시오.

Pruning

- $\text{if}((n - \text{cnt}) * 6 < m) \text{ return};$ 와 $\text{else if}((n - \text{cnt}) * 1 > m) \text{ return};$ 에서 가지 치기가 발생함
- 앞의 사례와 다르게 기저 조건에서 가지 치기를 수행

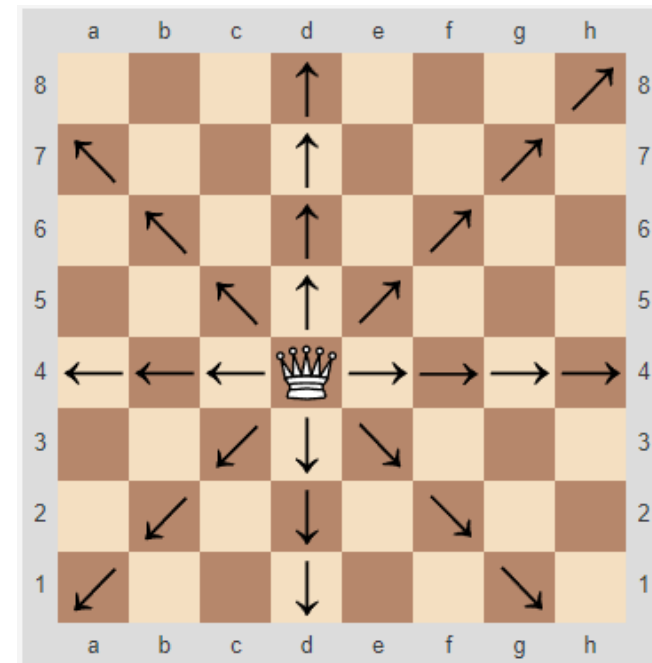
2. Backtracking

N-Queens

- $N * N$ 크기의 체스판 위에 N개의 퀸을 서로 공격할 수 없도록 배치하는 모든 경우의 수를 구하거나, 가능한 배치를 찾는 문제
- 퀸의 공격 범위는 같은 행, 같은 열, 같은 대각선

N-Queens 문제를 풀기 전에... (백트래킹 문제 아님)

- [4693 N Rook](#)



2. Backtracking

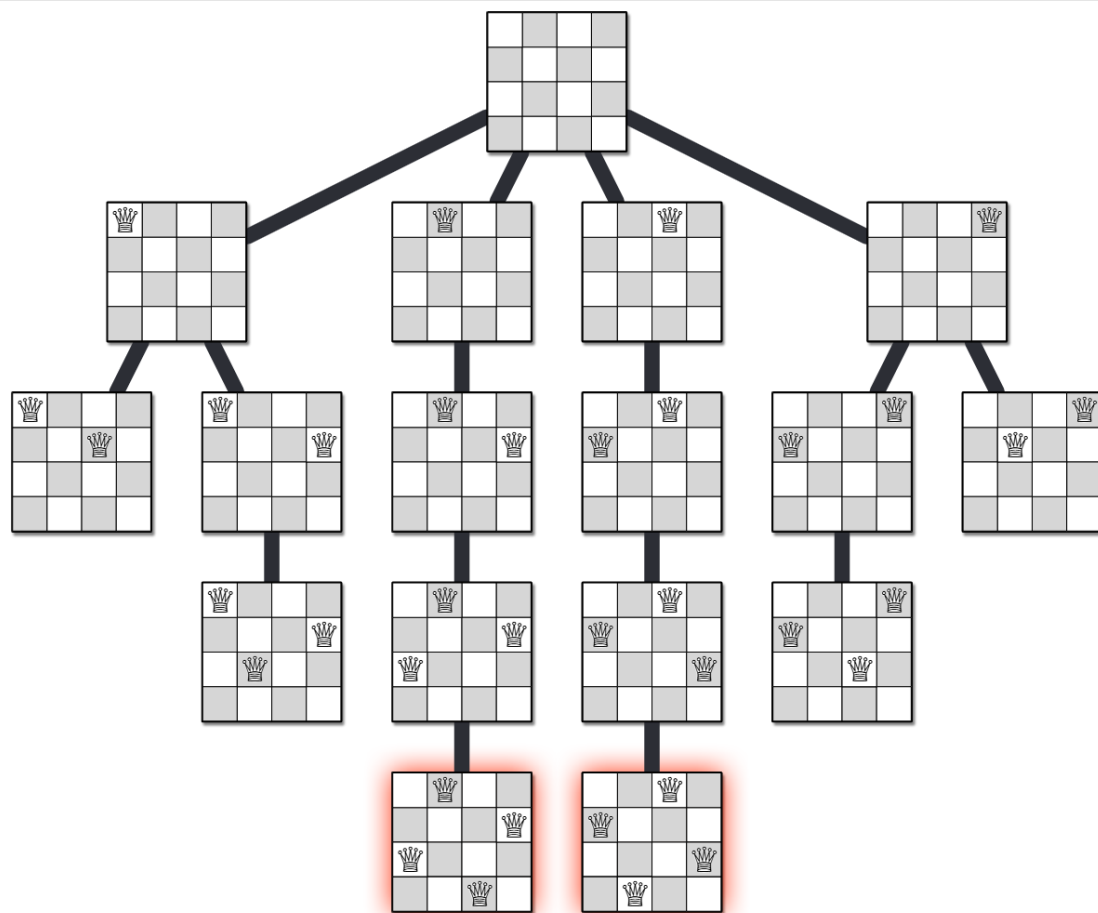


Figure 2.3. The complete recursion tree of Gauss and Laquière's algorithm for the 4 queens problem.

Idea

- 모든 칸에 퀸을 놓아보는 방법은 시간 초과
- 퀸은 한 행에 단 하나만 존재할 수 있으므로 1행부터 N행까지 차례로 퀸을 하나씩 배치
- 퀸을 배치했다면 다음 행으로 가서 새로운 퀸을 배치하되, 이전에 배치한 퀸의 공격범위와 충돌한다면 가지 치기

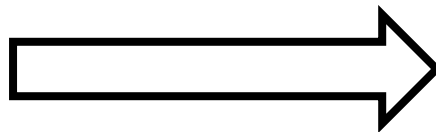
2. Backtracking

- 1889 N Queen
 - 2차원 배열 코드 // 시간 초과
 - 1차원 배열 코드

차원 압축

- 맞았습니다!! 를 받기 위해서는 차원 압축을 해야 함

0	1	0	0
0	0	0	1
1	0	0	0
0	0	1	0



2. Backtracking: 실습



실습

- [6058 N Rook 2](#)
 - [코드](#)
- [8129 몇 번째 조합](#)
 - [코드](#)
- [1027 좋은수열](#)